

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna
College of Computer Studies

ITEC 204 – Data Structures and Algorithms

Laboratory Activity 1: IT Automation Incident Ticket Manager

Name: Ryan Brando T. Si
Section: BSIT-2B
Repository: ITEC204-DSA-Si_Ryan

1. Program Title

IT Automation Incident Ticket Manager

2. Objective

The objective of this activity is to apply the concepts of linear data structures in a practical IT-related scenario. Specifically, this program aims to simulate how a support team manages incident tickets generated by automation bots by implementing basic operations such as adding, displaying, searching, removing, and counting records using a list.

3. Scenario

Automation bots run daily tasks such as generating reports, sending notifications, syncing data, and processing invoices. Sometimes these bots fail and generate an incident ticket that the IT support team has to check. Each ticket has an Incident ID, the name of the Bot that triggered it, and a short description of what went wrong. This program acts as a simple ticket manager so the support team (or in this case, the student testing it) can add new tickets, view them, look one up quickly, remove one once it's resolved, and see how many tickets are still active.

4. Data Structure Used

This program uses a **Python list** as its main linear data structure, named `tickets`. Each ticket is stored as a dictionary inside the list, with three keys: `id`, `bot`, and `description`. A list was chosen because it naturally supports the operations needed here — appending new tickets at the end, looping through to display or search, and removing a specific item — all with simple, built-in Python methods (`append()`, `remove()`) without needing a more complex structure.

5. Program Operations

- **Add:** Asks the user for an Incident ID, Bot name, and Short Description, checks that the ID isn't already used, then appends the new ticket to the list.
- **Display:** Loops through the list and prints every active ticket in a simple table format.
- **Search:** Asks for an Incident ID, loops through the list, and prints the matching ticket's details if found, or a not-found message if it isn't.
- **Remove:** Asks for an Incident ID, finds the matching ticket, and removes it from the list using `.remove()`, simulating a resolved ticket being closed out.
- **Count:** Uses Python's built-in `len()` function on the list to display the total number of currently active tickets.

All five operations are accessed through a single numbered menu (options 1–5), with an extra option 6 added to quickly load the 10 sample tickets for testing, and option 7 to exit the program.

6. Source Code

The full source code is also included separately as `incident_ticket_manager.py` inside the LAB EXER 1 folder of the repository. A copy is provided below for reference.

```
"""
ITEC 204 - Data Structures and Algorithms
Laboratory Activity 1: IT Automation Incident Ticket Manager

Data Structure Used: List (a linear data structure)
Each ticket is stored as a dictionary with keys: 'id', 'bot', 'description'
inside a Python list called `tickets`.

Operations implemented:
1. Add a new incident ticket
2. Display all active incident tickets
3. Search for a ticket by Incident ID
4. Remove a resolved ticket
5. Display the total number of active tickets
"""

# The list that holds all active incident tickets (our linear data structure)
tickets = []

# Sample data provided in the activity, used for quick testing
SAMPLE_TICKETS = [
    {"id": "INC1392939", "bot": "BOT-Inventory", "description": "Failed to generate the daily report"},
    {"id": "INC1392940", "bot": "BOT-Email", "description": "Failed to send the scheduled notification"},
    {"id": "INC1392941", "bot": "BOT-DataSync", "description": "Encountered an error during data transfer"},
    {"id": "INC1392942", "bot": "BOT-Invoice", "description": "Failed to process an invoice"},
    {"id": "INC1392943", "bot": "BOT-Report", "description": "Failed to generate the weekly report"},
    {"id": "INC1392944", "bot": "BOT-FileTransfer", "description": "Failed to upload the required file"},
    {"id": "INC1392945", "bot": "BOT-DataEntry", "description": "Encountered an error while entering records"},
    {"id": "INC1392946", "bot": "BOT-Backup", "description": "Failed to complete the scheduled backup"},
    {"id": "INC1392947", "bot": "BOT-Validation", "description": "Failed to validate the submitted records"},
    {"id": "INC1392948", "bot": "BOT-Notification", "description": "Failed to send the system alert"},
]

def add_ticket():
    """Adds a new incident ticket to the list."""
    print("\n--- Add New Incident Ticket ---")
    incident_id = input("Enter Incident ID: ").strip()

    # Prevent duplicate Incident IDs
    for ticket in tickets:
        if ticket["id"] == incident_id:
            print(f"Ticket {incident_id} already exists. Ticket not added.")
            return

    bot = input("Enter Bot name: ").strip()
    description = input("Enter Short Description: ").strip()

    new_ticket = {"id": incident_id, "bot": bot, "description": description}
    tickets.append(new_ticket)
    print(f"Ticket {incident_id} added successfully.")

def display_tickets():
    """Displays all active incident tickets."""
    print("\n--- Active Incident Tickets ---")
    if not tickets:
        print("No active incident tickets.")
        return

    print(f"{'Incident ID':<15}{'Bot':<20}{'Short Description'}")
    print("-" * 65)
```

```

        for ticket in tickets:
            print(f"{ticket['id']:<15}{ticket['bot']:<20}{ticket['description']}]}")

def search_ticket():
    """Searches for a specific ticket using its Incident ID."""
    print("\n--- Search Incident Ticket ---")
    incident_id = input("Enter Incident ID to search: ").strip()

    for ticket in tickets:
        if ticket["id"] == incident_id:
            print("Ticket found:")
            print(f"    Incident ID : {ticket['id']}")
            print(f"    Bot       : {ticket['bot']}")
            print(f"    Description : {ticket['description']}")
            return

    print(f"No ticket found with Incident ID: {incident_id}")

def remove_ticket():
    """Removes a resolved incident ticket using its Incident ID."""
    print("\n--- Remove Resolved Ticket ---")
    incident_id = input("Enter Incident ID to remove: ").strip()

    for ticket in tickets:
        if ticket["id"] == incident_id:
            tickets.remove(ticket)
            print(f"Ticket {incident_id} removed successfully.")
            return

    print(f"No ticket found with Incident ID: {incident_id}")

def count_tickets():
    """Displays the total number of active incident tickets."""
    print("\n--- Total Active Tickets ---")
    print(f"Total active incident tickets: {len(tickets)}")

def load_sample_data():
    """Loads the 10 sample tickets provided in the activity, for quick testing."""
    print("\n--- Loading Sample Data ---")
    loaded = 0
    for sample in SAMPLE_TICKETS:
        # Avoid duplicates if sample data is loaded more than once
        if not any(t["id"] == sample["id"] for t in tickets):
            tickets.append(sample.copy())
            loaded += 1
    print(f"{loaded} sample ticket(s) loaded.")

def print_menu():
    print("\n===== IT AUTOMATION INCIDENT TICKET MANAGER =====")
    print("1. Add a new incident ticket")
    print("2. Display all active incident tickets")
    print("3. Search for an incident ticket")
    print("4. Remove a resolved incident ticket")
    print("5. Display total number of active tickets")
    print("6. Load sample data (10 test tickets)")
    print("7. Exit")

def main():
    while True:
        print_menu()
        choice = input("Enter your choice (1-7): ").strip()

        if choice == "1":

```

```
        add_ticket()
    elif choice == "2":
        display_tickets()
    elif choice == "3":
        search_ticket()
    elif choice == "4":
        remove_ticket()
    elif choice == "5":
        count_tickets()
    elif choice == "6":
        load_sample_data()
    elif choice == "7":
        print("Exiting program. Goodbye!")
        break
    else:
        print("Invalid choice. Please enter a number from 1 to 7.")

if __name__ == "__main__":
    main()
```

7. Program Output

Below is a sample run of the program, showing the sample data being loaded, displayed, searched, one ticket removed, and the final count. Full screenshots of each operation are included in the screenshots folder inside LAB EXER 1.

```
===== IT AUTOMATION INCIDENT TICKET MANAGER =====
Enter your choice (1-7): 6

--- Loading Sample Data ---
10 sample ticket(s) loaded.

Enter your choice (1-7): 2

--- Active Incident Tickets ---
Incident ID      Bot              Short Description
-----
INC1392939      BOT-Inventory    Failed to generate the daily report
INC1392940      BOT-Email        Failed to send the scheduled notification
INC1392941      BOT-DataSync     Encountered an error during data transfer
INC1392942      BOT-Invoice      Failed to process an invoice
INC1392943      BOT-Report       Failed to generate the weekly report
INC1392944      BOT-FileTransfer Failed to upload the required file
INC1392945      BOT-DataEntry    Encountered an error while entering records
INC1392946      BOT-Backup       Failed to complete the scheduled backup
INC1392947      BOT-Validation   Failed to validate the submitted records
INC1392948      BOT-Notification Failed to send the system alert

Enter your choice (1-7): 3
Enter Incident ID to search: INC1392941
Ticket found:
  Incident ID : INC1392941
  Bot        : BOT-DataSync
  Description : Encountered an error during data transfer

Enter your choice (1-7): 4
Enter Incident ID to remove: INC1392945
Ticket INC1392945 removed successfully.

Enter your choice (1-7): 5

--- Total Active Tickets ---
Total active incident tickets: 9
```

8. Conclusion

Honestly, this activity looked scary at first because of all the GitHub folder stuff, but once I got the program itself working it wasn't that bad. Using a list for the tickets made sense once I thought about it — it's basically just a container I can keep adding to, loop through, and remove things from, which is exactly what a ticket system needs to do. The part I had to be careful with was the remove and search functions, since I had to make sure I was comparing the Incident ID correctly or it just wouldn't find anything. I also learned that testing with the sample data first (instead of typing everything by hand every time) saved me a lot of time, so I added a shortcut option in the menu just to load all 10 tickets at once. Overall, I understood better how linear data structures like lists work in a real, practical scenario instead of just a textbook example, and I feel more confident now going into the next lab exercises.